

การวิเคราะห์เชิงเปรียบเทียบด้านความมั่นคงปลอดภัยและประสิทธิภาพเครือข่ายของด็อกเกอร์คอนเทนเนอร์ กรณีศึกษาบริดจ์โหมดและโฮสต์โหมด

Comparative Analysis of Security and Network Performance in Docker Containers: A Case Study of Bridge Mode and Host Mode

กิตติชัย ฝั้นพรม

Kitichai Fanprom

ภาควิชาวิทยาการคอมพิวเตอร์

คณะวิทยาศาสตร์

มหาวิทยาลัยเชียงใหม่

Department of Computer Science,

Faculty of Science,

Chiang Mai University

Chiang Mai, Thailand

kitichai_f@cmu.ac.th

บทคัดย่อ — บทความนี้วิเคราะห์เปรียบเทียบสถาปัตยกรรมเครือข่ายของคอนเทนเนอร์ Docker ระหว่าง Bridge Mode และ Host Mode ในด้านความมั่นคงปลอดภัยตามหลักการ CIA Triad และด้านสภาพความพร้อมใช้งาน (Availability) การทดลองดำเนินการผ่านการจำลองดักจับและดัดแปลงข้อมูล ควบคู่กับการสร้างภาระงานระดับแอปพลิเคชัน (HTTP Stress) ภายใต้สภาพแวดล้อมที่ควบคุมด้วยการแยกแยะประมวลผล (Core Isolation) ผลการประเมินด้านความมั่นคงปลอดภัยพบว่า Host Mode มีความเสี่ยงจากการไม่มีระบบแยกส่วน ขณะที่ Bridge Mode สามารถสกัดกั้นการเข้าถึงโฮสต์หลักได้ แต่ยังคงเปราะบางต่อการโจมตีภายในเครือข่ายย่อยเดียวกัน ด้านประสิทธิภาพพบว่า ภายใต้สภาวะถูกโจมตี Bridge Mode มีอัตราการลดลงของปริมาณงานถึงร้อยละ 91.53 ซึ่งได้รับผลกระทบหนักกว่า Host Mode ที่ลดลงร้อยละ 88.62 อย่างมีนัยสำคัญทางสถิติ ($p\text{-value} < 0.01$) เนื่องจากเกิดคอขวดจากกระบวนการ NAT และ Connection Tracking ท้ายนี้ได้เสนอแนะให้พิจารณาใช้ Host Mode ในสภาพแวดล้อมปิดที่เชื่อถือเพื่องานที่ต้องการประสิทธิภาพสูง และใช้ Bridge Mode สำหรับงานทั่วไปเพื่อยกระดับความปลอดภัย

คำสำคัญ — ด็อกเกอร์, บริดจ์โหมด, โฮสต์โหมด, ความมั่นคงปลอดภัยเครือข่าย, ประสิทธิภาพเครือข่าย

ABSTRACT — This paper comparatively analyzes the network architectures of Docker containers, specifically Bridge Mode and Host Mode, evaluating their security based on the CIA Triad and performance in terms of availability. The experiments involved simulated attacks for data interception and modification, alongside application-level load generation (HTTP Stress) within a core-isolated environment. Security results reveal that Host Mode is vulnerable due to a lack of isolation, whereas Bridge Mode effectively prevents host-level access but remains susceptible to internal subnet attacks.

Regarding availability under attack conditions, Bridge Mode demonstrated a severe availability drop of 91.53%, suffering a significantly heavier impact than Host Mode's 88.62% drop ($p\text{-value} < 0.01$). This degradation is primarily attributed to the bottlenecks caused by NAT and connection tracking processes. The study recommends adopting Host Mode in highly trusted environments for high-performance tasks, while utilizing Bridge Mode for general applications to ensure robust security.

Keywords — Docker, Bridge Mode, Host Mode, Network Security, Network Performance

1. บทนำ

ปัจจุบันเทคโนโลยีคอนเทนเนอร์ (Containerization) อย่างด็อกเกอร์ (Docker) ได้รับความนิยมอย่างแพร่หลายในการให้บริการซอฟต์แวร์ อย่างไรก็ตาม สถาปัตยกรรมเครือข่าย (Network Architecture) ที่แตกต่างกันของ Docker ส่งผลกระทบบนตรงต่อทั้งประสิทธิภาพ (Performance) และความมั่นคงปลอดภัย (Security) โหมดเครือข่ายนิยมใช้งานทั่วไป ได้แก่ บริดจ์โหมด (Bridge Mode) ซึ่งอาศัยการแยกส่วนเครือข่าย (Network Isolation) ผ่าน Virtual Bridge และการทำ NAT (Network Address Translation) และโฮสต์โหมด (Host Mode) ที่แชร์ Network Stack ร่วมกับเครื่องหลัก (Host)

แม้ว่า Host Mode จะสามารถลดภาระการประมวลผล (Overhead) ของเครือข่ายได้ แต่การขาดกลไกแยกส่วนทรัพยากรอาจทำให้ระบบเสี่ยงต่อการถูกรบกวนจากภายใน (Lateral Movement) ในทางกลับกัน Bridge Mode มีจุดเด่นด้านความปลอดภัยจากการแยก

Namespace แต่อาจเกิดปัญหาคอขวด (Bottleneck) ที่ระดับเครือข่าย เมื่อระบบต้องรับภาระงานปริมาณมหาศาล

บทความนี้จึงมีวัตถุประสงค์เพื่อวิเคราะห์เปรียบเทียบสถาปัตยกรรมเครือข่ายระหว่าง Bridge Mode และ Host Mode โดยประเมินความมั่นคงปลอดภัยอย่างรอบด้านตามหลักการ CIA Triad ได้แก่ การรักษาความลับ (Confidentiality) และความถูกต้อง (Integrity) ผ่านการทดสอบดักจับและปลอมแปลงข้อมูลในเครือข่าย ควบคู่ไปกับการประเมินสภาพความพร้อมใช้งานของระบบ (Availability) ผ่านการจำลองสร้างภาระงานระดับแอปพลิเคชัน (HTTP Stress) อย่างต่อเนื่อง เพื่อหาข้อสรุปเชิงประจักษ์ด้านประสิทธิภาพการแยกส่วนเครือข่าย และนำเสนอแนวทางการเลือกใช้โหมดที่รักษาสมดุลระหว่างประสิทธิภาพและความปลอดภัย

2. ทฤษฎีและวรรณกรรมที่เกี่ยวข้อง

การประเมินสถาปัตยกรรมเครือข่ายของคอนเทนเนอร์จำเป็นต้องพิจารณาความสัมพันธ์ระหว่างประสิทธิภาพการทำงานและความมั่นคงปลอดภัยซึ่งมีทฤษฎีที่เกี่ยวข้องดังนี้

2.1. สถาปัตยกรรมเครือข่ายของ Docker

- Bridge Mode: มีการแยกส่วนเครือข่ายชัดเจน แต่ข้อมูลต้องผ่านกระบวนการ NAT ผ่านสวิตช์เสมือน ทำให้เกิด Overhead
- Host Mode: ใช้ Network Stack ร่วมกับโฮสต์หลักโดยตรง ไม่มีการทำ NAT ทำให้ได้ประสิทธิภาพสูงสุด แต่อาจมีความเสี่ยงจากการไม่แยกส่วนของทรัพยากรเครือข่าย

2.2. การประเมินความพร้อมใช้งานภายใต้ภาระงานระดับแอปพลิเคชัน

การทดสอบสภาพความพร้อมใช้งาน อาศัยการจำลองภาระงานระดับแอปพลิเคชัน (Application Layer หรือ OSI Layer 7) ผ่านโปรโตคอล HTTP เพื่อประเมินขีดจำกัดของ Network Layer (OSI Layer 3) และ Transport Layer (OSI Layer 4) เนื่องจากทุกคำขอ (Request) ใน Bridge Mode ต้องเข้าคิวผ่านกระบวนการ NAT การจำลองภาระงาน Layer 7 ปริมาณมหาศาลจึงเป็นการกดดันเครือข่ายโดยตรง เพื่อวัดประสิทธิภาพและคอขวดของกลไก Network Isolation

2.3. ความมั่นคงปลอดภัยตามหลักการ CIA Triad และความพยายามในการโจมตี

ความมั่นคงปลอดภัยของระบบเครือข่ายสามารถประเมินได้จากสามด้านหลัก ได้แก่ Confidentiality และ Integrity ซึ่งเกี่ยวข้องกับการป้องกันความเสี่ยงจากการดักจับข้อมูลหรือการปลอมแปลงข้อมูล ในขณะที่ Availability ขึ้นอยู่กับระดับการแยกส่วนของเครือข่ายและความพยายามที่ผู้โจมตีต้องใช้ในการรบกวนระบบ (Attacking Effort)

บทความนี้จึงทำการจำลองการโจมตีเพื่อประเมิน Availability ในระดับ Application Layer ผ่านการทดสอบ HTTP Stress ซึ่งเป็นการสร้างการเชื่อมต่อ (Connection) แบบขนานจำนวนมากที่ใช้ทรัพยากรระบบสูง เมื่อใช้งาน Bridge Mode ภาระงานระดับ Application Layer จะถูกส่งต่อไปยังระดับเครือข่ายและทรานสปอร์ต (Layer 3 และ Layer 4) ทำให้ระบบต้องจัดการกับกลไก Connection Tracking และกระบวนการ NAT สำหรับทุกแพ็กเก็ตที่ผ่านเครือข่าย

ดังนั้น การจำลองโหลดในลักษณะนี้จึงเป็นการทดสอบความทนทานของกลไกการแยกส่วนเครือข่าย (Isolation Mechanism) ต่อ Attacking

Effort ของผู้โจมตี โดยเปรียบเทียบกับ Host Mode ซึ่งตรงไปจากจากการโจมตีสามารถเข้าสู่ระบบหลักได้โดยตรงโดยไม่ผ่านกระบวนการ NAT

2.4. งานวิจัยที่เกี่ยวข้อง

การศึกษาของ Suo และคณะ [1] ยืนยันว่ากระบวนการ NAT และการจัดการแพ็กเก็ตผ่านสวิตช์เสมือนใน Bridge Mode คือสาเหตุหลักของคอขวดที่ทำให้ Throughput ลดลงและค่าความหน่วง (Latency) สูงขึ้นเมื่อเทียบกับ Host Mode สอดคล้องกับบทความของ Lee และคณะ [2] ที่พิสูจน์ว่าโครงสร้างคอนเทนเนอร์มีความเปราะบางต่อการโจมตีจากภายใน (Internal Attacks) ซึ่งส่งผลกระทบโดยตรงต่อทรัพยากรระบบและความพร้อมใช้งาน บทความนี้จึงต่อยอดโดยการนำตัวแปรด้านประสิทธิภาพมาทดสอบภายใต้สภาวะที่มี Attacking Effort สูง เพื่อหาข้อสรุปเชิงเปรียบเทียบ

3. วิธีการทดลอง

การทดลองนี้จำลองสภาพแวดล้อมระบบเครือข่ายเพื่อเปรียบเทียบสถาปัตยกรรม Bridge Mode และ Host Mode โดยมีรายละเอียดการดำเนินการดังนี้

3.1. สภาพแวดล้อมการทดลอง

การทดลองดำเนินการบนคอมพิวเตอร์พกพา HP Victus 16 ซึ่งติดตั้งหน่วยประมวลผลกลาง AMD Ryzen™ 7 5800H (8 cores, 16 threads) พร้อมหน่วยความจำหลักขนาด 32 GB DDR4 (Kingston Fury) และอุปกรณ์จัดเก็บข้อมูลแบบ NVMe M.2 SSD ขนาด 1 TB รุ่น WD Black SN770 ระบบปฏิบัติการหลักที่ใช้ในการทดลองคือ Parrot OS 7.1

สภาพแวดล้อมของคอนเทนเนอร์ใช้ Docker Engine version 26.1.5 สำหรับจำลองโครงสร้างระบบเครือข่ายและบริการเว็บ โดยแบ่งออกเป็นสองคอนเทนเนอร์ ได้แก่ เครื่องเป้าหมาย (Victim) ซึ่งรันบริการ Nginx version 1.29.4 ทำหน้าที่เป็นเว็บเซิร์ฟเวอร์ที่พอร์ต 8080 และเครื่องผู้โจมตี (Attacker) ซึ่งพัฒนาจาก Docker image ของ Kali Linux ที่มีการปรับแต่งเพิ่มเติมในเครื่องท้องถิ่น (Local Build) เพื่อใช้สำหรับสร้างภาระงานโจมตี HTTP

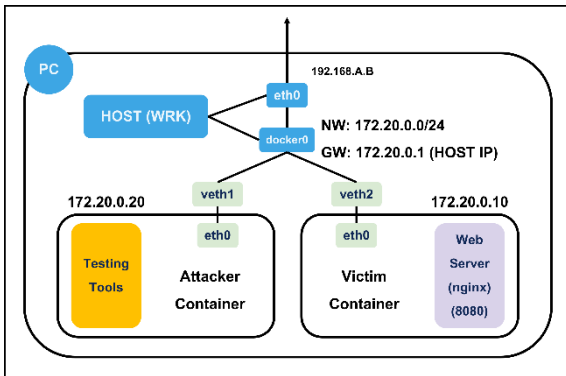
เพื่อป้องกันความคลาดเคลื่อนจากการจัดตารางการทำงานของระบบปฏิบัติการ (OS Scheduler) ผู้ทดลองจึงทำการแยกแกนประมวลผล (Core Isolation) ในระดับเคอร์เนลผ่านการกำหนดพารามิเตอร์ GRUB (isolcpus=4-11) โดยกำหนดให้คอนเทนเนอร์เครื่องเป้าหมายใช้ Logical CPU แกนที่ 4-7 ขณะที่คอนเทนเนอร์ผู้โจมตีใช้แกนที่ 8-11 นอกจากนี้ยังมีการจำกัดหน่วยความจำของแต่ละคอนเทนเนอร์ไม่เกิน 6 GB เพื่อควบคุมปัจจัยด้านทรัพยากรระบบให้คงที่ตลอดการทดลอง (ดังรูปที่ 1.)

การทดสอบภาระงานระดับ Application Layer ใช้เครื่องมือ wrk version 4.1.0 เพื่อสร้าง HTTP stress ในรูปแบบการเชื่อมต่อแบบขนานจำนวนมาก โดยทำการเปรียบเทียบประสิทธิภาพของ Docker network ระหว่าง Bridge Mode และ Host Mode ภายใต้สภาวะการโจมตี

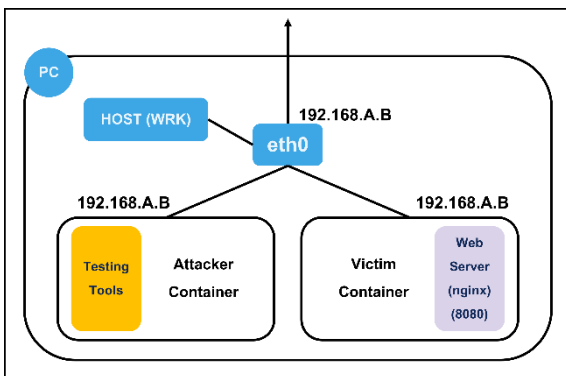
ในส่วนของการวิเคราะห์กราฟฟิคเครือข่าย เครื่องผู้โจมตีติดตั้งเครื่องมือ tcpdump version 4.99.5, ngrep version 1.47.1 และ arpspoof version 2.5a2 สำหรับตรวจสอบและวิเคราะห์แพ็กเก็ตในระดับเครือข่ายระหว่างการทดลอง

3.2. สถาปัตยกรรมเครือข่ายที่ใช้ในการทดสอบ

- Bridge Mode: เครื่องเป้าหมายใช้เลข ที่อยู่ไอพี (IP Address) 172.20.0.20 และเครื่องผู้โจมตีใช้ IP Address 172.20.0.10 ถูกจัดให้อยู่ในเครือข่ายย่อยเดียวกัน (172.20.0.0/24) โดยข้อมูลต้องวิ่งผ่าน Virtual Bridge (docker0) และเกตเวย์ (Gateway) ที่ใช้ IP Gateway 172.20.0.1 (ดังรูปที่ 1)
- Host Mode: ทุกคอนเทนเนอร์แยกการแยกส่วน Namespace และใช้งาน Network Stack ร่วมกับอินเทอร์เฟซเครือข่าย (Network Interface) ของโฮสต์หลักโดยตรง (ดังรูปที่ 2)



รูปที่ 1. สถาปัตยกรรมเครือข่ายแบบ Bridge Mode



รูปที่ 2. สถาปัตยกรรมเครือข่ายแบบ Host Mode

3.3. วิธีการจำลองภาระงานและการโจมตี

ดำเนินการจำลองการโจมตีตามหลักการ CIA Triad ดังนี้

- Confidentiality: ดำเนินการตรวจสอบการรั่วไหลของข้อมูล HTTP โดยใน Host Mode อาศัยการดักจับแพ็กเก็ตบนอินเทอร์เฟซของโฮสต์ด้วยเครื่องมือ tcpdump ส่วนใน Bridge Mode ใช้เทคนิคการปลอมแปลงตารางเส้นทาง (ARP Spoofing) ด้วยเครื่องมือ arpspoof ร่วมกับการดักอ่านแพ็กเก็ตด้วย ngrep
- Integrity: ดำเนินการทดสอบการเข้าถึงและดัดแปลงบริการภายใน (Internal Service) โดยจำลองการส่งpayloadอันตราย (Malicious Payload) ผ่านโปรโตคอล HTTP ด้วยคำสั่ง curl -X POST ไปยังพอร์ตเป้าหมาย (พอร์ต 9000 สำหรับ Host Mode และ 55555 สำหรับ Bridge Mode) เพื่อประเมินว่าสามารถเขียนทับไฟล์ของระบบได้สำเร็จหรือไม่
- Availability: ดำเนินการจำลองภาระงานระดับ Application Layer โดยใช้เครื่องมือ wrk โดยเริ่มจากการวัดค่าในสภาวะปกติ (Baseline)

จากนั้นทำการสร้างภาระงานโจมตีด้วยพารามิเตอร์ 4 Threads, 200 Connections เป็นเวลา 300 วินาที พร้อมกับบันทึกการวัดผลซ้ำจำนวน 5 รอบ รอบละ 30 วินาที และพักระบบต่อไปอีก 20 วินาที เพื่อเปรียบเทียบประสิทธิภาพก่อนและระหว่างการถูกโจมตี

3.4. วิธีการวัดผลและการวิเคราะห์ข้อมูล

ตัวชี้วัดประสิทธิภาพหลักที่ใช้ในการประเมินประกอบด้วย ปริมาณงาน (Requests/sec) และความหน่วงโดยนำผลลัพธ์เชิงปริมาณมาคำนวณหาอัตราการลดลงของสภาพความพร้อมใช้งาน (Availability Drop) ตามสมการ

$$Availability Drop (\%) = \frac{R_B - R_A}{R_B} \times 100 \quad (1)$$

กำหนดให้ R คือค่าเฉลี่ยของปริมาณงาน (Requests/sec), R_B คือค่าเฉลี่ยของปริมาณงานในสภาวะปกติ (Baseline) และ R_A คือค่าเฉลี่ยของปริมาณงานภายใต้สภาวะถูกโจมตี (Under Attack)

นอกจากนี้ การศึกษาในครั้งนี้ได้นำผลลัพธ์มาวิเคราะห์เปรียบเทียบเชิงสถิติ (T-Test) เพื่อประเมินระดับนัยสำคัญของการเปลี่ยนแปลงประสิทธิภาพ ระหว่างสภาวะปกติและสภาวะถูกโจมตีในทั้งสองสถาปัตยกรรมเครือข่าย

4. ผลการทดลอง

การเปรียบเทียบประสิทธิภาพและความมั่นคงปลอดภัยระหว่างสถาปัตยกรรม Bridge Mode และ Host Mode ได้ผลลัพธ์ดังนี้

4.1. ผลการทดสอบด้าน Confidentiality

- Host Mode: การที่คอนเทนเนอร์แชร์ Network Stack ร่วมกับโฮสต์หลัก ทำให้ผู้โจมตีสามารถใช้เครื่องมือ tcpdump ดักจับ (Sniffing) การสื่อสารของพอร์ต 8080 บนทุกอินเทอร์เฟซ (Interfaces) ได้โดยตรง ส่งผลให้ข้อมูล HTTP ถูกเปิดเผยทั้งหมด
- Bridge Mode: แม้กลไก Network Namespace จะแยกวงเครือข่ายออกจากโฮสต์หลัก แต่เมื่อผู้โจมตีอยู่ในเครือข่ายย่อยเดียวกัน ผู้โจมตีสามารถใช้เทคนิค ARP Spoofing หลอกหลวงทิศทางการส่งข้อมูล ทำให้สามารถดักอ่านข้อมูล HTTP ด้วย ngrep ได้สำเร็จเช่นกัน

4.2. ผลการทดสอบด้าน Integrity

- Host Mode: ผู้โจมตีมีสิทธิ์เข้าถึงบริการภายใน (Localhost) ของโฮสต์หลักผ่านพอร์ต 9000 และสามารถส่ง Payload ผ่าน HTTP POST ไปเขียนทับ (Overwrite) ไฟล์บริการระบบได้สำเร็จโดยตรง ซึ่งแสดงให้เห็นถึงความเสี่ยงจากการไม่มีระบบแยกส่วน
- Bridge Mode: ผู้โจมตีไม่สามารถเข้าถึงบริการบนโฮสต์หลักได้เนื่องจากถูกสกัดกั้นโดยกฎไฟร์วอลล์ (Firewall Rules) ของโฮสต์ อย่างไรก็ตาม ผู้โจมตียังคงสามารถส่ง Payload เข้าโจมตีบริการภายในของคอนเทนเนอร์เป้าหมายและเขียนทับไฟล์ระบบได้สำเร็จ

4.3. ผลการทดสอบด้าน Availability

การประเมินสภาพความพร้อมใช้งานพิจารณาจากค่าเฉลี่ยปริมาณงาน (Requests/sec) และความหน่วงจากการทดสอบจำนวน 5 รอบ ผลการทดลองเปรียบเทียบสภาวะปกติและสภาวะถูกโจมตีได้ดังนี้

ตารางที่ 1. ผลการทดสอบเฉลี่ยด้าน Availability

ประเภท การทดสอบ \ โหมด	Bridge Mode	Host Mode
ปริมาณงานสภาวะปกติ (Req/sec)	27,142.56	30,102.69
ปริมาณงานขณะถูกโจมตี (Req/sec)	2,300.13	3,426.48
ความหน่วงเฉลี่ยขณะโจมตี (ms)	8.75	6.34
Availability Drop (%)	91.53	88.62

จากตารางที่ 1. ในสภาวะปกติ Host Mode มีประสิทธิภาพพื้นฐานสูงกว่า Bridge Mode เนื่องจากการลดภาระของการแปลงที่อยู่เครือข่าย (NAT) เมื่อเข้าสู่สภาวะถูกโจมตีด้วยภาระงานระดับ Application Layer ด้วย HTTP Stress ประสิทธิภาพของทั้งสองสถาปัตยกรรมลดลงอย่างมีนัยสำคัญ อย่างไรก็ตาม Bridge Mode มีอัตราการสูญเสียความพร้อมใช้งาน (Availability Drop) สูงกว่า Host Mode (91.53% เทียบกับ 88.62%) และมีความหน่วงเฉลี่ยสูงกว่า (8.75 ms เทียบกับ 6.34 ms) ผลลัพธ์นี้ชี้ให้เห็นว่า กลไกการแยกส่วนเครือข่ายด้วย Virtual Bridge และกระบวนการ Connection Tracking ของ iptables ใน Bridge Mode จะกลายเป็นคอขวด (Bottleneck) ที่รุนแรงขึ้นเมื่อต้องรับมือกับความพยายามในการโจมตี (Attacking Effort) ปริมาณมหาศาล

ผลการวิเคราะห์เปรียบเทียบความแตกต่างของค่าเฉลี่ยด้วยสถิติ T-Test (Independent Samples t-test) พบผลลัพธ์ที่สำคัญ 2 ประการ ดังนี้

- ปริมาณงาน (Throughput) ในสภาวะถูกโจมตีของทั้ง Bridge Mode และ Host Mode ลดลงจากสภาวะปกติอย่างมีนัยสำคัญทางสถิติที่ระดับ .01 (p-value < 0.01) ยืนยันว่าการโจมตีแบบจำลองภาระงาน (HTTP Stress) ส่งผลกระทบต่อสภาพความพร้อมใช้งานอย่างชัดเจน
- เมื่อเปรียบเทียบระหว่างโหมดเครือข่ายในสภาวะที่ถูกโจมตี พบว่า Host Mode สามารถรองรับปริมาณงานเฉลี่ย (3,426.48 Req/sec) ได้สูงกว่า Bridge Mode (2,300.13 Req/sec) อย่างมีนัยสำคัญทางสถิติที่ระดับ .01 (p-value < 0.01) ข้อมูลทางสถิตินี้ยืนยันสมมติฐานที่ว่า ภาระการประมวลผล (Overhead) จากกระบวนการ NAT ใน Bridge Mode เป็นปัจจัยสำคัญที่ทำให้เครือข่ายเกิดคอขวดและสูญเสียประสิทธิภาพเร็วกว่า Host Mode เมื่อต้องรับมือกับความพยายามในการโจมตี (Attacking Effort) ปริมาณมหาศาล

5. สรุปและอภิปรายผล

5.1. สรุปผลการวิจัย

การศึกษานี้ได้วิเคราะห์เปรียบเทียบสถาปัตยกรรมเครือข่ายของคอนเทนเนอร์ระหว่าง Bridge Mode และ Host Mode ภายใต้หลักการความมั่นคงปลอดภัย (CIA Triad) และการประเมินประสิทธิภาพสภาพความพร้อมใช้งาน (Availability) ผลการทดสอบเชิงประจักษ์สรุปได้ว่า โหมดเครือข่ายทั้งสองมีจุดเด่นและข้อจำกัดที่สวนทางกันอย่างชัดเจน

Host Mode ในด้าน Confidentiality และ Integrity มีความเปราะบางสูงสุด เนื่องจากการใช้ Network Stack ร่วมกับโฮสต์หลัก ทำให้ผู้โจมตี

สามารถดักจับข้อมูลบนอินเทอร์เฟซทั้งหมด และเข้าถึงบริการภายใน (Localhost) ของโฮสต์ได้โดยตรง ในขณะที่ Bridge Mode มีกลไกแยกส่วน (Namespace) ที่ช่วยป้องกันการโจมตีข้ามไปยังโฮสต์หลักได้อย่างเด็ดขาด แต่ยังคงมีความเสี่ยงต่อการถูกโจมตีแบบดักจับและดัดแปลงข้อมูลผ่านเทคนิค ARP Spoofing ภายในเครือข่ายย่อย (Subnet) เดียวกัน ในด้าน Availability จากการจำลองภาระงานระดับแอปพลิเคชัน (HTTP Stress) ปริมาณมหาศาล พบว่า Bridge Mode มีอัตราการลดลงของปริมาณงาน (Availability Drop) สูงถึง 91.53% และมีความหน่วงเฉลี่ย 8.75 ms ซึ่งได้รับผลกระทบหนักกว่า Host Mode ที่มีอัตราการลดลง 88.62% และความหน่วงเฉลี่ย 6.34 ms

5.2. อภิปรายผล

จากผลการทดลอง สามารถนำมาอภิปรายเชื่อมโยงกับทฤษฎีสถาปัตยกรรมเครือข่ายได้ดังนี้

5.2.1. ข้อขัดข้องของกระบวนการแปลงเลขที่อยู่ของเครือข่าย อัตราการสูญเสียความพร้อมใช้งานที่สูงกว่าของ Bridge Mode เกิดจากกลไกการสื่อสารที่ต้องผ่านสวิตช์เสมือน (docker0) และการกรองแพ็กเก็ตผ่าน iptables เมื่อระบบเผชิญกับการจำลองคอนเนกชันแบบขนาน (Concurrent connections) ปริมาณมหาศาลในระดับ Application Layer หรือ Layer 7 ระบบเครือข่ายระดับเคอร์เนลจะต้องทำการจดจำสถานะการเชื่อมต่อ (Connection Tracking) และทำ NAT ในทุกๆ แพ็กเก็ตเกิด Overhead หรือ ภาระการประมวลผลที่สะสมนี้จึงกลายเป็นคอขวดที่ทำให้ Throughput ลดลงอย่างรวดเร็วและเพิ่มความหน่วงให้กับระบบ

5.2.2. ความสมดุลระหว่างประสิทธิภาพและการแยกส่วน Host Mode แสดงให้เห็นถึงความทนทานต่อภาระงานเครือข่ายที่ดีกว่า เนื่องจากทราฟฟิกสามารถสื่อสารผ่านอินเทอร์เฟซเครือข่ายจริง (eth0) ได้โดยตรงโดยไม่ต้องผ่านกระบวนการจำลองเครือข่าย ทำให้ประหยัดทรัพยากรการประมวลผล อย่างไรก็ตาม ประสิทธิภาพที่เพิ่มขึ้นนี้ต้องแลกมาด้วยความเสี่ยงด้านความมั่นคงปลอดภัยระดับวิกฤตจากการขาดสมรรถนะการแยกส่วนทรัพยากร

5.2.3. ข้อจำกัดของการทดลอง การทดสอบนี้ดำเนินการบนเครื่องคอมพิวเตอร์พกพาเพียงเครื่องเดียว (Local Environment) โดยอาศัยการทำ Core Isolation เพื่อควบคุมปัจจัยแทรกซ้อนจากระบบปฏิบัติการ ดังนั้น ค่าความหน่วงที่วัดได้จึงสะท้อนถึงภาระการประมวลผลหรือ Overhead ของซอฟต์แวร์เครือข่าย (Software Stack) ภายในระบบปฏิบัติการเป็นหลัก โดยไม่ได้รวมถึงความหน่วงแฝงทางกายภาพ (Physical Network Latency) ที่อาจเกิดขึ้นในระบบเครือข่ายจริง อย่างไรก็ตาม สภาพแวดล้อมจำลองนี้เพียงพอต่อการควบคุมตัวแปร เพื่อพิสูจน์ความแตกต่างของประสิทธิภาพกลไกเครือข่ายของ Docker ทั้งสองโหมด

5.3. ข้อเสนอแนะ

5.3.1. แนวทางการเลือกใช้งาน หากแอปพลิเคชันมีความจำเป็นต้องประมวลผลทราฟฟิกเครือข่ายปริมาณมหาศาลและต้องการความหน่วงต่ำสุด (เช่น ระบบประมวลผลแบบ Real-time) การพิจารณาใช้ Host Mode เป็นทางเลือกที่เหมาะสม แต่ต้องดำเนินการในสภาพแวดล้อมเครือข่ายปิดที่ได้รับความไว้วางใจระดับสูง (Trusted Environment) เท่านั้น และตั้งค่า Firewall Rules อย่างรัดกุมเหมาะสมแล้วเท่านั้น สำหรับแอปพลิเคชันและเว็บเซิร์ฟเวอร์ทั่วไป ควรเลือกใช้ Bridge Mode

หรือ Custom Bridge เสมอ เพื่อยกระดับความปลอดภัยผ่านการแยกส่วน (Isolation) พร้อมทั้งตั้งค่า Firewall Rules อย่างรัดกุม

5.3.2. การเสริมความมั่นคงปลอดภัยใน Bridge Mode เพื่อลดทอนความเสี่ยงจากการโจมตีภายในควรมีการตั้งค่าจำกัดการสื่อสารระหว่างคอนเทนเนอร์ (ICC = false) การทำ Static ARP เพื่อป้องกันการทำ ARP Spoofing รวมถึงการจำกัดทรัพยากร (Resource Limits) ควบคู่ไปกับการติดตั้งระบบป้องกันระดับแอปพลิเคชัน (เช่น WAF หรือ Rate Limiting) เพื่อป้องกันไม่ให้งานระดับ Layer 7 กดดันเครือข่ายจนสูญเสียสภาพความพร้อมใช้งาน

เอกสารอ้างอิง

- [1] X. Suo, Y. Zhao, G. Chen and J. Rao, "An Analysis and Empirical Study of Container Networks," in *Proceedings of IEEE INFOCOM 2018*, Honolulu, 2018.
- [2] H. Lee, S. Kwon and J.-H. Lee, "Experimental Analysis of Security Attacks for Docker Container Communications," *Electronics*, vol. 12, no. 4, 2023.